CONTROLE DE VERSÃO COM GIT NA LINHA DE COMANDO

*Sete arquivos no diretório: `firewall.sh`, `.bak`, `.bak2`, `.old`, `.funciona`, `.FINAL` e `.FINAL2` — nenhum com data confiável, porque um `cp` preservou o carimbo e outro não. A regra que bloqueava o acesso indevido estava num deles, e **o que estava em produção não era ele.***

095
GUIAS LINUX

TRÊS ÁREAS E UM REMOTO: O MAPA QUE RESOLVE QUASE TODA DÚVIDA

DIRETÓRIO DE TRABALHO — os arquivos que você edita

`git add →` / `← git restore`

ÍNDICE (*STAGE*) — o próximo commit sendo montado

`git commit →` / `← restore --staged`

REPOSITÓRIO LOCAL — histórico completo em `.git/`

`git push →` / `← git fetch`

REMOTO — origin, outra máquina

`git pull = git fetch + git merge` — do remoto direto ao seu diretório de trabalho

O índice existe para você escolher *o que* entra no próximo commit. Editou cinco arquivos e só três são a correção? `git add` nos três, `git commit`, e os outros dois seguem esperando — é o que permite um histórico em que cada commit é *uma* mudança coerente. **Os três `diff` seguem o mesmo mapa:** `git diff` = trabalho×índice; `--staged` = índice×último commit; `HEAD` = trabalho×último commit — confundi-los é a origem nº 1 de "o Git não vê a minha mudança". **E a propriedade que muda tudo:** depois de entrar num commit é quase impossível perder algo — o `git reflog` guarda por onde o `HEAD` passou nos últimos 90 dias. **Antes do primeiro `git add`, não há rede nenhuma.**

DESFAZER: O MAPA DOS ARREPENDIMENTOS

QUERO DESFAZER...	COMANDO · EFEITO
uma edição minha	`git restore arq` — volta ao que está no índice
um `git add`	`git restore --staged arq` — sai do índice, edição intacta
a mensagem do último commit	`git commit --amend` — reescreve o último commit
commits ainda locais	`git reset --soft HEAD~1` — desfaz o commit, mantém tudo
commits locais e o trabalho	`git reset --hard HEAD~1` — **destrutivo**: perde as edições
um commit já publicado	`git revert <hash>` — cria um commit que inverte o outro

kali@kali: ~/scripts

```
$ git reset --hard HEAD~1 # o commit "sumiu"
$ git reflog | head -3
9f2c1ab HEAD@{0}: reset: moving to HEAD~1
4b7e3d0 HEAD@{1}: commit: experimento
$ git reset --hard HEAD@{1} # voltou
# conflito de merge: decisão humana, não erro
<<<<<<< HEAD
iptables -A INPUT --dport 8080 -j DROP
=======
nft add rule inet filter input tcp dport 8080 drop
>>>>>>> migra-nftables
# edite deixando o final correto, apague as 3 marcas,
# git add + git commit — ou git merge --abort
```

Duas regras evitam quase todo estrago. 1. `--amend` e `reset` *reescrevem* o histórico; `revert` não. Enquanto o commit é só seu, reescreva à vontade; **depois do `push`, use `revert`** — reescrever o que já foi publicado quebra o repositório de todo mundo que já baixou. **2.** Perdeu-se e não sabe onde está? `git reflog` lista as posições recentes do `HEAD` e `git reset --hard HEAD@{3}` volta a uma delas — o `reflog` é a rede de segurança, mas **só do que chegou a um commit**: o que `git restore` ou `reset --hard` descartam some de verdade.

NA PRÁTICA — DO PRIMEIRO COMMIT AO /ETC VERSIONADO

kali@kali: ~ — o ciclo inteiro

```
$ git config --global user.name "Seu Nome"
$ git config --list --show-origin | head
# mostra QUAL arquivo definiu cada valor:
# /etc/gitconfig < ~/.gitconfig < .git/config
$ git init && git add firewall.sh
$ git diff --staged # confira ANTES de commitar
$ git commit -m "firewall: bloqueia 8080, que
      expunha o painel"  # a mensagem diz o PORQUÊ
$ git switch -c ajuste; git merge ajuste
$ git fetch; git log --oneline main..origin/main
# fetch só baixa; pull já mescla. Em máquina
# compartilhada, fetch+log é o hábito prudente.
$ sudo apt install etckeeper && sudo etckeeper init
# cada apt install vira um commit em /etc
```

.GITIGNORE — DESDE O PRIMEIRO COMMIT

```
*.log  *.tmp  *.bak  __pycache__/
.venv/  segredos.env  *.key  *.pem  id_rsa*
```

A mensagem de commit explica o *porquê*, não o *quê* — o `diff` já mostra o que mudou. "Corrige bug" não ajuda ninguém em seis meses. **Autenticação por SSH, sempre:** senha em URL HTTPS fica em texto claro no `.git/config` e vaza no primeiro `cat`. **O `etckeeper` transforma "o que mudou na configuração quando aquele pacote entrou?" num `git log`** — o antídoto contra o diretório de `.bak` da abertura. Mas repositório de **`/etc` contém segredos por definição** (`shadow`, chaves de serviço, tokens): o `/etc/.git` fica `0700` e **não** vai para remoto compartilhado. Versionar `/etc` é ótimo para auditoria local; publicá-lo é vazar a máquina inteira.

Um segredo commitado é um segredo vazado, e apagá-lo depois não resolve. Chave privada, token ou senha que entrou num commit fica no histórico **permanentemente**: um `git rm` no commit seguinte tira o arquivo do estado atual e mantém o conteúdo acessível em todo clone que já existe. Se foi para um remoto, considere a credencial comprometida de fato e **rotacione-a imediatamente** — é a única ação que realmente resolve. O `git filter-repo` limpa o repositório para o futuro, mas não recupera o que já foi copiado. Prevenção: `.gitignore` desde o primeiro commit e `git diff --staged` antes de cada `git commit`.

RESUMO RÁPIDO

- ✓ **O Git substitui o diretório de `.bak` por um histórico com autor, data e motivo** — vale para script, configuração e dotfile, não só para código.
- ✓ **Três áreas explicam quase tudo:** trabalho, índice e repositório local. `add`, `commit` e `push` vão; `restore`, `restore --staged` e `fetch` voltam.
- ✓ **Reescrever histórico local é livre; publicado, não.** `reset` e `--amend` só antes do `push`; depois disso, `revert`.
- ✓ **O `reflog` recupera quase tudo que passou por um commit** — e nada do que nunca foi commitado.
- ✓ **Segredo commitado é segredo vazado:** `.gitignore` desde o início, `diff --staged` antes de cada commit, rotação imediata se escapou.
- ✓ **Conflito não é erro**, é o Git dizendo que a decisão é humana — resolva e `git add`, ou `git merge --abort` e pense melhor.

NO KALI

O Git já vem instalado e é dependência de metade do ecossistema: boa parte das ferramentas fora dos repositórios oficiais se instala com `git clone` — e clonar repositório de terceiro é **trazer código de terceiro** que roda na sua máquina assim que você o executa. Leia o que se clona antes de rodar e o cuidado do *hardening* aplicado ao código-fonte. **Os ganchos do Git executam código automaticamente, e isso é vetor real:** os arquivos em `.git/hooks/` disparam em commit e merge. O `git clone` **não** copia os hooks do remoto — mas um `post-checkout` num repositório que alguém te entregou compactado roda na primeira operação; `ls -la .git/hooks/` antes é hábito barato. **Versionar os scripts e as anotações de um teste autorizado é rastreabilidade** — cada commit tem autor e carimbo, e responde "quem fez o quê e quando". O que **não** entra: credenciais capturadas, dados pessoais do alvo e material sensível colhido no teste, que têm tratamento previsto em contrato e não pertencem a um `git push`. **Segredo em repositório é um dos achados mais comuns em revisão de segurança**, e o Kali traz `trufflehog` e `gitleaks` para varrer o histórico inteiro atrás de chaves e tokens — rodados contra os **seus** repositórios, ou os do cliente dentro do escopo autorizado. **Cuidado com `git init` no home inteiro:** ele captura `.ssh/`, `.zsh_history`, `.msf4/`. Versionar dotfiles é ótimo, mas a abordagem correta lista o que *incluir*. **Fora do Debian:** Git é Git e o `etckeeper` está no EPEL; a diferença é a *versão* — em RHEL estável, `git switch` e `git restore` (que chegaram na 2.23) podem não existir, e aí valem `git checkout -b` e `git checkout --`.